Name: Tuguinay, Nanelle

Section: 402I

```
# ============================================================
# COMPONENT III. AUTOMATED EXPERIMENT WITH SIDE-BY-SIDE EXCEL FORMAT
# ============================================================

# 1. INSTALL REQUIRED LIBRARIES
!pip install -q faiss-cpu sentence-transformers transformers sentenc

# 2. IMPORT LIBRARIES
from google.colab import files
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np
import pandas as pd
import re
import os
import openpyxl
from openpyxl.styles import Font, PatternFill, Alignment, Border, Si
from openpyxl.utils.dataframe import dataframe_to_rows
from openpyxl.utils import get_column_letter

print("=" * 70)
print("UPLOAD DOCUMENT (VAW_dataset.txt)")
print("=" * 70)

uploaded = files.upload()
if not uploaded:
    raise RuntimeError("No file was uploaded.")

txt_file = next(iter(uploaded.keys()))
with open(txt_file, "r", encoding="utf-8") as f:
    raw_text = f.read()

# Sanitize raw text and clean OCR artifacts
raw_text = re.sub(r'Fourteenth Congress.*?assembled::\s*', '', raw_t
raw_text = raw_text.replace("dime of conquest", "time of conquest")
raw_text = raw_text.replace("pregnant;-", "pregnancy")
raw_text = raw_text.replace("include. but", "include, but")
raw_text = re.sub(r';\s*-\s*', ' ', raw_text)

cleaned_text = re.sub(r"\s+", " ", raw_text).strip()
pages = [{"page": 1, "text": cleaned_text}]

# 3. CHUNKING FUNCTION
```

```
def create_chunks(pages, chunk_size, chunk_overlap, min_tail_length=
    chunks = []
    chunk_id = 0
    step_size = chunk_size - chunk_overlap
    if step_size <= 0:
        step_size = 1
    for page_data in pages:
        page_number = page_data["page"]
        page_text = page_data["text"]
        start = 0
        page_chunk_positions = []
        while start < len(page_text):
            remaining = len(page_text) - start
            if remaining < min_tail_length and page_chunk_positions:
                prev_pos = page_chunk_positions[-1]
                prev_chunk = chunks[prev_pos]
                prev_start = prev_chunk["_start"]
                prev_chunk["text"] = page_text[prev_start:].strip()
                break
            end = min(start + chunk_size, len(page_text))
            chunk_text = page_text[start:end].strip()
            if chunk_text:
                chunks.append({
                    "chunk_id": chunk_id,
                    "page": page_number,
                    "text": chunk_text,
                    "_start": start
                })
                page_chunk_positions.append(len(chunks) - 1)
                chunk_id += 1
            start += step_size
    for chunk in chunks:
        chunk.pop("_start", None)
    return chunks

# 4. DEFINE 12 EXPERIMENTAL CONFIGURATIONS (C1 - C12)
configs = [
    {"config": "C1", "size": 300, "overlap": 0},
    {"config": "C2", "size": 300, "overlap": 30},
    {"config": "C3", "size": 300, "overlap": 60},
    {"config": "C4", "size": 500, "overlap": 0},
    {"config": "C5", "size": 500, "overlap": 50},
    {"config": "C6", "size": 700, "overlap": 0},
    {"config": "C7", "size": 700, "overlap": 140},
    {"config": "C8", "size": 1000, "overlap": 0},
    {"config": "C9", "size": 1000, "overlap": 200},
    {"config": "C10", "size": 1200, "overlap": 300},
    {"config": "C11", "size": 1500, "overlap": 0},
    {"config": "C12", "size": 1500, "overlap": 300},
]

# 5. DEFINE 9 VERIFIED TEST QUESTIONS (3 Easy, 3 Average, 3 Difficul
questions = [
    {"id": "Q1", "difficulty": "Easy", "question": "What is the offi
```

```
    {"id": "Q2", "difficulty": "Easy", "question": "What percentage
    {"id": "Q3", "difficulty": "Easy", "question": "What does CEDAW
    {"id": "Q4", "difficulty": "Average", "question": "What specific
    {"id": "Q5", "difficulty": "Average", "question": "What is the t
    {"id": "Q6", "difficulty": "Average", "question": "What special
    {"id": "Q7", "difficulty": "Difficult", "question": "Which gover
    {"id": "Q8", "difficulty": "Difficult", "question": "What specif
    {"id": "Q9", "difficulty": "Difficult", "question": "What are th
]

# 6. RUN EXPERIMENT LOOP
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

detailed_logs = []
config_results = []
diff_records = []

print("\nRunning chunking experiment across C1-C12 configurations...
for cfg in configs:
    chunks = create_chunks(pages, cfg["size"], cfg["overlap"])
    chunk_texts = [c["text"] for c in chunks]

    vectorizer = TfidfVectorizer().fit(chunk_texts + [q["question"]
    chunk_mat = vectorizer.transform(chunk_texts)

    p3_list, r3_list, mrr_list = [], [], []
    diff_metrics = {"Easy": {"p": [], "r": [], "m": []},
                    "Average": {"p": [], "r": [], "m": []},
                    "Difficult": {"p": [], "r": [], "m": []}}

    for q in questions:
        q_vec = vectorizer.transform([q["question"]])
        sims = cosine_similarity(q_vec, chunk_mat).flatten()
        top_indices = sims.argsort()[::-1][:3]

        relevant_count = 0
        first_rel_rank = 0

        for rank, idx in enumerate(top_indices, start=1):
            chunk_txt = chunks[idx]["text"]
            is_rel = any(kw.lower() in chunk_txt.lower() for kw in q
            if is_rel:
                relevant_count += 1
                if first_rel_rank == 0:
                    first_rel_rank = rank

            detailed_logs.append({
                "Configuration": cfg["config"],
                "Question ID": q["id"],
                "Difficulty": q["difficulty"],
                "Expected evidence": ", ".join(q["keywords"]),
                "Retrieved rank": rank,
                "Retrieved chunk ID": chunks[idx]["chunk_id"],
```

```
                "Source reference": f"Page 1, Chunk {chunks[idx]['ch
                "Relevant?": "Yes" if is_rel else "No",
                "Evidence covered": q["id"] if is_rel else "None"
            })

        p3 = relevant_count / 3.0
        r3 = 1.0 if relevant_count > 0 else 0.0
        mrr = (1.0 / first_rel_rank) if first_rel_rank > 0 else 0.0

        p3_list.append(p3)
        r3_list.append(r3)
        mrr_list.append(mrr)

        diff_metrics[q["difficulty"]]["p"].append(p3)
        diff_metrics[q["difficulty"]]["r"].append(r3)
        diff_metrics[q["difficulty"]]["m"].append(mrr)

    config_results.append({
        "Config": cfg["config"],
        "Chunk size": cfg["size"],
        "Overlap": cfg["overlap"],
        "Total chunks": len(chunks),
        "Precision@3": round(np.mean(p3_list), 3),
        "Recall@3": round(np.mean(r3_list), 3),
        "MRR@3": round(np.mean(mrr_list), 3)
    })

    for diff in ["Easy", "Average", "Difficult"]:
        d_p = diff_metrics[diff]["p"]
        d_r = diff_metrics[diff]["r"]
        d_m = diff_metrics[diff]["m"]
        diff_records.append({
            "Configuration": cfg["config"],
            "Difficulty": diff,
            "Precision@3": round(np.mean(d_p), 3) if d_p else 0.0,
            "Recall@3": round(np.mean(d_r), 3) if d_r else 0.0,
            "MRR@3": round(np.mean(d_m), 3) if d_m else 0.0
        })

# 7. GENERATE FORMATTED SIDE-BY-SIDE EXCEL WORKBOOK
wb_final = openpyxl.Workbook()
wb_final.remove(wb_final.active)

df_a = pd.DataFrame(config_results)
df_b = pd.DataFrame(diff_records)
df_log = pd.DataFrame(detailed_logs)

# Sheet 1: Table A
ws_a = wb_final.create_sheet(title="Table A - Config Results")
for r in dataframe_to_rows(df_a, index=False, header=True):
    ws_a.append(r)

# Sheet 2: Table B
ws_b = wb_final.create_sheet(title="Table B - By Difficulty")
```

```
for r in dataframe_to_rows(df_b, index=False, header=True):
    ws_b.append(r)

# Sheet 3: Table C (Side-by-Side PIVOT)
ws_c = wb_final.create_sheet(title="Table C - Side-by-Side Log")

header_fill = PatternFill(start_color="365F91", end_color="365F91",
header_font = Font(name="Calibri", size=10, bold=True, color="FFFFFF
sub_header_fill = PatternFill(start_color="DCE6F1", end_color="DCE6F
sub_header_font = Font(name="Calibri", size=10, bold=True, color="00
thin_border = Border(
    left=Side(style='thin', color='D9D9D9'),
    right=Side(style='thin', color='D9D9D9'),
    top=Side(style='thin', color='D9D9D9'),
    bottom=Side(style='thin', color='D9D9D9')
)

configs_list = df_log['Configuration'].unique()
columns_per_block = 9
spacing_columns = 1

for pair_idx in range(0, len(configs_list), 2):
    row_offset = (pair_idx // 2) * 32 + 1
    current_configs = configs_list[pair_idx : pair_idx + 2]

    for c_idx, cfg_name in enumerate(current_configs):
        col_start = c_idx * (columns_per_block + spacing_columns) +
        cfg_df = df_log[df_log['Configuration'] == cfg_name].copy()

        ws_c.merge_cells(start_row=row_offset, start_column=col_star
        title_cell = ws_c.cell(row=row_offset, column=col_start, val
        title_cell.fill = header_fill
        title_cell.font = header_font
        title_cell.alignment = Alignment(horizontal="center", vertic

        headers = ["Config", "Question ID", "Difficulty", "Expected
        for h_idx, h_text in enumerate(headers):
            cell = ws_c.cell(row=row_offset + 1, column=col_start +
            cell.fill = sub_header_fill
            cell.font = sub_header_font
            cell.alignment = Alignment(horizontal="center", vertical
            cell.border = thin_border

        for r_idx, (_, row) in enumerate(cfg_df.iterrows()):
            data_row = row_offset + 2 + r_idx
            ws_c.cell(row=data_row, column=col_start + 0, value=row[
            ws_c.cell(row=data_row, column=col_start + 1, value=row[
            ws_c.cell(row=data_row, column=col_start + 2, value=row[
            ws_c.cell(row=data_row, column=col_start + 3, value=str(
            ws_c.cell(row=data_row, column=col_start + 4, value=row[
            ws_c.cell(row=data_row, column=col_start + 5, value=row[
            ws_c.cell(row=data_row, column=col_start + 6, value=row[
            ws_c.cell(row=data_row, column=col_start + 7, value=row[
            ws_c.cell(row=data_row, column=col_start + 8, value=str(
```

```
            for h_idx in range(columns_per_block):
                c = ws_c.cell(row=data_row, column=col_start + h_idx
                c.border = thin_border
                c.font = Font(name="Calibri", size=9)
                c.alignment = Alignment(horizontal="center" if h_idx

for sheet in wb_final.worksheets:
    for col in sheet.columns:
        col_letter = get_column_letter(col[0].column)
        sheet.column_dimensions[col_letter].width = 15

excel_output_filename = "VAW_experiment_results.xlsx"
wb_final.save(excel_output_filename)

print(f"\nExperiment complete! File '{excel_output_filename}' genera
files.download(excel_output_filename)
```

```
======================================================================
UPLOAD DOCUMENT (VAW_dataset.txt)
======================================================================
```

Browse... No files selected. Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving VAW_dataset.txt to VAW_dataset (4).txt

Running chunking experiment across C1-C12 configurations...

Experiment complete! File 'VAW_experiment_results.xlsx' generated ar
```